T A R E Q O

09 — Tests

Le projet a **une couverture de tests dans chaque couche**, tirant parti de la testabilité qu'offre la Clean Architecture. Les tests s'exécutent sur la VM (pas d'émulateur) — y compris ceux de SQLite.

Version 2.1.3+23 · 14 septembre 2026

Table des matières

Comment exécuter 3

Structure 3

Stratégie par couche 4

- Domaine (cas d'usage) 4
- Données (datasources/repositories SQLite) 4
- Présentation (écrans) 4

Patterns et conventions 5

Harnais SQL/RLS/RPC (`scripts/sql_check.sh`) 5

Documents connexes 6

Comment exécuter

```
# Tous les tests (440+ verts — Aug 9)
flutter test
# Un fichier spécifique
flutter test test/domain/usecases/process_sale_use_case_sqlite_test.dart
# Avec rapport de couverture
flutter test --coverage
```

CI : .github/workflows/ci.yml exécute analyze → test+coverage (seuil 45%) → build APK du flavor dev → `sql_check.sh` contre un vrai service Postgres 16 (le harnais SQL/RLS/RPC — voir plus bas).

SQLite sur la VM Windows : le loader de package:sqlite3 retombe automatiquement sur la winsqlite3.dll système (via AppDatabase.configureLoader — une sonde défensive dans test/helpers/test_sqlite.dart). Aucune configuration manuelle requise.

Structure

```
test/
 core/
  database/app_database_test.dart # Schéma, transactions réentrantes, migrations W4/W11
  utils/quantity_test.dart # Milli-unités ↔ unités (W11)
  utils/scale_barcode_test.dart # Parseur 20PPPPPWWWWW + normalisation PLU (W11)
  security/
  secure_storage_service_test.dart # Backend en mémoire
  secure_supabase_local_storage_test.dart # Session sécurisée + stockage PKCE (S1, Aug 6)
 data/
  datasources/local/
   product_sqlite_data_source_test.dart
   sale_sqlite_data_source_test.dart
   stock_lot_sqlite_data_source_test.dart # Ordre FIFO + portée magasin (W11)
   purchase_sqlite_data_source_test.dart # Achats avec péremption/lot (W11)
   financial_transaction_sqlite_data_source_test.dart
   debt_sqlite_data_source_test.dart
   sqlite_unit_of_work_test.dart
  repositories/ # Repositories (avec fakes/impls en mémoire)
  product_repository_impl_test.dart
  sale_repository_impl_test.dart
  financial_transaction_repository_impl_test.dart
  debt_repository_impl_test.dart
  auth_repository_impl_test.dart
 domain/
  entities/ # financial_transaction_test, sale_test
  usecases/
  add_debt_use_case_test.dart
  add_financial_transaction_use_case_test.dart
  get_cash_flow_use_case_test.dart
  get_pending_debts_use_case_test.dart
  get_settled_debts_use_case_test.dart
  product_use_cases_test.dart
  settle_debt_use_case_test.dart
  process_sale_use_case_test.dart # avec des fakes
```

```
 process_sale_use_case_sqlite_test.dart # avec une vraie base en mémoire
 fifo_stock_use_case_test.dart # Diminution de lots FIFO (W11)
 add_purchase_use_case_test.dart # Entrée en stock avec péremption (W11)
helpers/test_sqlite.dart # openTestDatabase() : sqlite en mémoire + schéma
presentation/
 controllers/expiry_alerts_provider_test.dart # Alertes rouge/jaune (W11)
 screens/
 login_screen_test.dart
 app_shell_test.dart
 product_list_screen_test.dart
 new_sale_sheet_test.dart
 sales_screen_test.dart
 sale_detail_screen_test.dart
 financial_screen_test.dart
 add_debt_sheet_test.dart
 debts_screen_test.dart
widget_test.dart # Dashboard de bout en bout avec overrides (racine test/)
```

Stratégie par couche

Domaine (cas d'usage)

- Fakes de contrats (IProductRepository, ISaleRepository, IDebtRepository, IFinancialTransactionRepository, IUnitOfWork) — pas de serveur, pas de base de données ;
- Couvre chaque exception du domaine (InvalidPriceException, InsufficientStockException, InvalidPeriodException, DebtNotFoundException, idempotence du règlement…) ;
- `process_sale_use_case_sqlite_test.dart` teste le flux réel avec openTestDatabase() (base en mémoire avec le schéma de l'app) — validant l'atomicité (rollback en cas d'échec) et l'intégrité ;
- `fifo_stock_use_case_test.dart` couvre la diminution FIFO (lot à péremption la plus proche d'abord, surplus passant au lot suivant, sans péremption → par date d'entrée), increase créant des lots et la pré-validation du stock avant toute mutation (pas de diminution partielle).

Données (datasources/repositories SQLite)

- test/helpers/test_sqlite.dart :

```
Database openTestDatabase() {
AppDatabase.configureLoader();
final db = sqlite3.openInMemory();
db.execute('PRAGMA foreign_keys = ON;');
AppDatabase.createSchema(db);
return db;
}
```

- Teste le SQL, le mapping des lignes (enums par nom, dates en millis epoch, bool 0/1), les CHECK du schéma et SqliteUnitOfWork (BEGIN/COMMIT/ROLLBACK, file FIFO) ;
- stock_lot_sqlite_data_source_test.dart valide l'ordre FIFO de la requête (expiry ASC → entry ASC), la portée par magasin et la mise à jour atomique de current_quantity ; quantity_test.dart et scale_barcode_test.dart couvrent la math des milli-unités et le parseur 20PPPPPWWWWW (incl. la normalisation des PLU).

Présentation (écrans)

- Providers surchargés via ProviderScope(overrides: [...]) — sans toucher à GetIt ni aux datasources ;
- Exemple (test/widget_test.dart) : simule une session active (authStateProvider.overrideWith(Stream.value(AppUser(...)))), des fakes de cas d'usage et vérifie les KPI + la navigation de la coquille ;
- Pattern : await tester.pumpAndSettle() + expect(find.text(...), findsOneWidget) ;
- expiry_alerts_provider_test.dart valide la classification des alertes (périmé = rouge, < 30 jours = jaune) et l'ordre d'urgence.

Patterns et conventions

Pattern	Détail
Fakes via implements	Les classes _Fake... implémentent l'interface du domaine (ex. : _FakeGetProductsUseCase implements GetProductsUseCase)
Overrides de providers	productsProvider.overrideWithValue(...), authStateProvider.overrideWith(...)
Base de test	SQLite en mémoire (sqlite3.openInMemory) avec le vrai schéma, sans seed
Atomicité	Les tests de ProcessSaleUseCase valident le rollback complet quand une étape échoue
Immuabilité	Les datasources renvoient des listes immuables (les tests vérifient les comportements de copie/tri)

Harnais SQL/RLS/RPC (scripts/sql_check.sh)

Exécute sqlcheck_stub.sql (stubs d'auth) + supabase/schema.sql contre un PostgreSQL jetable, puis en tant que rôle non superutilisateur (app_user, reflétant l'authenticated de Supabase) :

1. le schéma s'applique proprement ;
2. le rôle super admin se résout via la table public.super_admins (C3 — jamais l'e-mail du JWT) ;
3. le RLS isole par auth.uid() avec triggers anti-falsification ;
4. l'accès des employés est limité au magasin (is_my_store, C2) ;
5. les RPC d'argent atomiques s'exécutent pour de vrai (process_sale, register_debt, settle_debt, process_purchase) avec effets vérifiés — y compris la date de réalisation estampillée au règlement (Aug 6), les cas négatifs (stock insuffisant) et les ajouts W11 : diminution de lots FIFO (péremption la plus proche d'abord), création de lots à l'achat (avec péremption facultative) et l'invariant `stock = Σ lots`.

De plus, `scripts/sql_check_live.sh` est la vérification live en lecture seule (31 checks, dont 7 W11 : colonnes, stock_lots/RLS/index, corps des RPC et invariant des lots) — utilisée par le runbook de migration W11.

Exécution locale (Docker) : PGPASSWORD=... bash scripts/sql_check.sh [PGHOST] [PGUSER] [PGDATABASE] (Attention : destructif — base jetable uniquement).

Documents connexes

- Architecture — pourquoi les cas d'usage sont testables avec des mocks.
- Guide du développeur — commandes et configuration.
- Flux métier — ce que valident les tests de vente/crédit/caisse.